Name: Jonathan Ramjattan, Ondrea Wagner, Dorien Parris

Project Proposal: MelodyBank

Problem Statement

Modern music producers often experience "melodic block." While audio loops (WAV/MP3) are common, they are "locked" as a producer cannot easily change the notes, the key, or the synth sound without losing quality. Existing MIDI collections are often sold in bloated, expensive packs. There is no centralized, searchable marketplace for individual, high-quality MIDI sequences that producers can audition and customize before buying.

Target Users

- **Loopmakers:** Composers and music theorists who want to sell their original chord progressions and lead lines.
- **Music Producers:** Beat-makers using DAWs (Ableton, FL Studio, Logic) who need flexible MIDI patterns to trigger their own software instruments.

Value Proposition

MelodyHub offers **dynamic previews**. Unlike audio stores, MelodyHub uses a web-based synthesizer to "perform" the MIDI files in the browser. This allows users to hear the melody and potentially switch preview sounds (e.g., Piano vs. Synth) before downloading the raw MIDI data.

Minimum Viable Product (MVP)

- **MIDI Upload & Storage:** Sellers can upload `.mid` files with metadata (Key, Scale, Complexity).
- **Web-Audio Preview:** A frontend synthesizer that parses and plays MIDI files.
- **Piano Roll Visualization:** A visual grid showing the notes of the MIDI file.
- **Search & Filter:** Find patterns by "Type" (Chords, Arps, Basslines) or "Key."
- **Personal MIDI Vault:** A dashboard for buyers to access their purchased MIDI files.

External API Integration

API: Spotify Web API

- **Data Provided:** Global genre trends and artist-specific audio features.
- **Use Case:** The system will use Spotify's "Genre" and "Artist" endpoints to help sellers categorize their MIDI. For example, a seller can tag a MIDI file as "Drake-style" or "Lofi

Hip Hop," and the API will validate these tags against current trending Spotify genres to ensure the MIDI appears in relevant searches.

System Decomposition

System Modules

1. **Auth Module:** Secure login and profile management for Creators and Buyers.
2. **MIDI Processing Engine:** A frontend module using **Tone.js** and **@tonejs/midi** to parse binary MIDI data and trigger web-synthesizers.
3. **Marketplace Module:** Handles the CRUD operations for MIDI listings, including search filters.
4. **Library Module:** Manages the relationship between users and their purchased/saved MIDI assets.

Data Flow Diagram (Level 0)

User (Creator) → Upload MIDI (.mid) → Spring Boot API → PostgreSQL / S3 Storage
User (Buyer) ← JSON MIDI Data ← React Frontend (Tone.js Synth) ← Spring Boot API
React Frontend ↔ Spotify API (For Tagging/Trends)

Entities / Data Objects

- **User:** `id`, `username`, `email`, `credits`, `is_creator`
- **MidiSequence:** `id`, `title`, `file_url`, `key_signature`, `tempo_bpm`, `category` (Chords/Lead), `creator_id`
- **Relationship:** One **User** (Creator) can have many **MidiSequences**. A **Purchase** table links **Users** to the **MidiSequences** they have unlocked.

Architecture Diagram

- **Frontend:** React.js + Tone.js (For MIDI synthesis and Piano Roll rendering).
- **Backend:** Spring Boot (Managing metadata, file links, and user permissions).
- **Database:** PostgreSQL (Storing user data and MIDI metadata).
- **Storage:** Cloudinary or AWS S3 (Hosting the `.mid` files).
-

Demo-Centric Planning

Scenario: "From Audition to DAW"

1. **Search:** The user filters for "Neo-Soul Chords" in "E-Flat Minor."

2. **Interactive Preview:** The user clicks play. The **React Piano Roll** lights up, and a **Tone.js Synth** plays the MIDI. The user toggles a "Preview Instrument" button to hear how the chords sound on a Rhodes piano versus a Lead synth.
3. **Acquisition:** The user adds the MIDI to their library.
4. **Verification:** The user clicks "Download," receiving the `.mid` file ready to be dragged into their DAW.

Responsible AI Usage Log

AI Tool	Prompt	Purpose	Influence on Design
Gemini	"How to play MIDI in a React browser app?"	Technical Feasibility	Confirmed that **Tone.js** is the industry standard for web-audio MIDI synthesis.

Project Planning (Roles)

- **Lead Architect (Backend):** Spring Boot setup, API endpoints for MIDI metadata, and Spotify API integration.
- **Audio Engineer (Frontend):** Tone.js implementation, MIDI parsing, and the interactive Piano Roll UI.
- **Database & Security:** PostgreSQL schema design, JWT Authentication, and file storage logic.

GitHub Repository: https://github.com/JonathanRamjattan/MelodyBank-COP3060